

类与对象

面向对象

| 抽象、封装、继承、多态

类

类成员的可访问范围

private（默认）：成员函数

public：任何地方

protected：可以被成员函数和子类的成员函数访问

注：在类的成员函数内部，可访问：

1. 当前对象的全部属性、函数
2. 同类其它对象的全部属性、函数

构造函数

名字与类名相同、可以有参数、不能有返回值

作用是初始化对象

默认生成无参构造函数（若自己定义了则不生成）

对象生成时自动被调用

可以有多个（参数个数**or**类型不同）

最好是**public**的，**private**不能直接用于初始化

```
class Rectangle {
private: int width, height;
public:
    Rectangle() : width(0), height(0) { cout << "调用了无参构造函数" << endl; }
    Rectangle(int side) : width(side), height(side) { cout << "调用了单参数构造函数" << endl; }
    Rectangle(int w, int h) : width(w), height(h) { cout << "调用了双参数构造函数" << endl; }
    void display() { cout << "宽: " << width << ", 高: " << height << endl; }
};
```

构造函数在数组中的使用

```
class test{
    public:
        test(int n){} //(1)
        test(int n,m){} //(2)
        test(){} //(3)
};
test array[3]={1,test(1,2)}; //(1)(2)(3)
```

复制构造函数

参数一定要是引用！！

只有一个参数

形如X::X(X &) 或X::X(const X &)(前者不能以常量对象作为参数)

生成默认复制构造函数、完成复制功能

复制构造函数起作用的三种情况

1. 用一个类的对象**初始化**同类的另一个对象

```
Complex c1;
Complex c2(c1);
Complex c2=c1; //初始化，不是赋值
```

2. 调用了**参数是类A的对象**的函数

```
void Func(A a1){}
```

3. **返回值**是类A的对象

```
A Func(){
    A b(4);
    return b;
}
```

#对象间赋值并不调用复制构造函数

类型转换构造函数

#只有一个参数、且不是复制构造函数的构造函数

```

class A{
    int v;
public:
    A(){}
    A(int i):v(i){} //(1)
    explicit A(string i){v=stoi(i);} //(2)
}

int main(){
    A a1;
    A a2=3; //隐式调用(1)
    a2=5;
    A a3=A(3) //显式调用(1)
    A a4="4" //隐式调用(2) ✗
    A a5=A("98") //显式调用(2)
    a5=A("11")
}

```

1. (1)(2)均为类型转换构造函数，其中(2)为显式类型转换构造函数，需要显式调用，不能自动转换。
2. 类型转换构造函数在**赋值**中也会用到

析构函数

~类名

没有参数和返回值

一个类至多一个

缺省析构函数什么也不做(若自己定义了则不生成)

```

class string{
private:
    char * p;
public:
    string(){
        p=new char[10];
    }
    ~string(){
        delete [] p;
    }
}

```

析构函数的调用

1. 对象数组生命周期结束后，对象数组的**每个元素**的析构函数都会被调用
2. delete运算导致析构函数调用（局部/全局变量在离开作用域后自动调用析构函数，但是**new生成的对象不会自动调用析构函数**）
3. 析构函数在对象作为函数返回值返回后被调用

构造函数与析构函数的调用顺序

C++ 构造与析构顺序速查表

场景类型	构造顺序 (Construction)	析构顺序 (Destruction)	记忆口诀
局部对象	按代码定义的先后顺序	按定义的 相反 顺序	后进先出 (LIFO)
成员变量	按类中 声明 的顺序	按声明的 相反 顺序	先有砖头后有房
继承关系	基类 → 派生类	派生类 → 基类	先打地基再盖楼
静态对象	首次执行到定义处时	程序结束（main 退出）时	寿终正寝
多重继承	按继承列表的先后顺序	按继承列表的 相反 顺序	从左到右构造

静态成员

本质上为全局成员

比较维度	普通成员变量 (Instance Member)	静态成员变量 (Static Member)
关键字	无需特殊关键字	需要 <code>static</code> 修饰
归属	属于每个具体对象	属于整个类，由所有对象共享
存储位置	随对象存储（通常在栈或堆上）	全局/静态存储区
内存份数	每个对象各有一份独立拷贝	整个类仅存一份，不随对象增减（ 哪怕一个对象都不存在也存在 ）
生命周期	随对象创建而生，随对象销毁而亡	程序启动时分配，程序结束时释放
是否占对象大小	✅ 计入 <code>sizeof(对象)</code>	❌ 不计入 <code>sizeof(对象)</code>
初始化方式	构造函数、初始化列表或类内赋初值	类外定义并初始化（ <code>const static</code> 整型除外）
声明/定义位置	类内声明并分配空间	类内声明，类外定义（C++17 可用 <code>inline static</code> ）
类外访问方式	必须通过 <code>对象.成员</code> 或 <code>指针->成员</code>	类名:: <code>成员</code> (推荐) 或 <code>对象.成员</code>
this 指针	✅ 可通过 <code>this-></code> 访问	❌ 与 <code>this</code> 无关，不依赖具体对象

比较维度	普通成员变量 (Instance Member)	静态成员变量 (Static Member)
静态函数访问权	✗ 静态成员函数不可直接访问	✓ 静态成员函数可以访问
普通函数访问权	✓ 可以访问	✓ 可以访问
不完整类型支持	✗ 成员类型不能是所属类本身	✓ 可以是所属类类型（常用于单例模式）
典型用途	描述单个对象的特有属性（如姓名）	对象计数、共享配置、单例模式、类级常量
继承行为	派生类对象拥有基类成员的拷贝	基类与所有派生类共享同一个静态变量
访问控制	受 public/protected/private 限制	同样受 public/protected/private 限制

```
//代码示例
class Student {
public:
    std::string name;           // 普通成员：每个学生各有一个名字
    static int totalCount;      // 静态成员：全班共享一个计数器

    Student(const std::string& n) : name(n) { ++totalCount; }
    Student(const Student & other):name(other.name){++totalCount;}
    ~Student() { --totalCount; }
};

// 类外定义并初始化（必须）
int Student::totalCount = 0;

int main() {
    Student a("Tom"), b("Jerry");
    // a.name 和 b.name 各自独立
    // Student::totalCount == 2, 两个对象共享
}
```

成员对象与封闭类

封闭类

有成员对象的类叫做封闭类

注：封闭类的成员函数**不能**访问成员对象的私有成员

封闭类的构造函数

必须通过初始化列表完成对成员对象的初始化

```
class Engine {
    int horsepower;
public:
    Engine(int hp) : horsepower(hp) {
        cout << "Engine构造 hp=" << hp << endl;
    }
    ~Engine() { cout << "Engine析构" << endl; }
};

class Tire {
    int size;
public:
    Tire(int s) : size(s) {
        cout << "Tire构造 size=" << s << endl;
    }
    ~Tire() { cout << "Tire析构" << endl; }
};

// ★ Car 就是封闭类（包含了 Engine 和 Tire 对象）
class Car {
    Engine engine;    // 对象成员1
    Tire tire;        // 对象成员2
    int price;
public:
    // 必须通过初始化列表来初始化对象成员
    Car(int hp, int s, int p) : engine(hp), tire(s), price(p) {
        cout << "Car构造" << endl;
    }
    ~Car() { cout << "Car析构" << endl; }
};
```

封闭类构造函数与析构函数的执行顺序

1. 对象成员的构造先于封闭类自身的构造函数体执行
2. 多个对象成员之间按声明顺序构造（与初始化列表书写顺序无关！）
3. 析构顺序与构造顺序完全相反

封闭类的复制构造函数

若封闭类的对象是用默认复制构造函数初始化的，那么包含的成员对象也会用复制构造函数初始化

友元

1. 友元可以分为**友元函数**和**友元类**两种
2. 友元类之间的关系**不能传递、不能继承**
3. 友元是**单向的**

友元函数

一个类的友元函数可以访问该类的**私有成员**

可以将一个类的成员函数（**包括构造、析构函数**）or全局函数说明为另一个类的友元即

友元函数不是这个类的成员函数

```
#include <iostream>
using namespace std;

class Box; // 前向声明

// =====
// Printer 类（其成员函数将作为 Box 的友元）
// =====
class Printer {
public:
    void print(const Box& b); // 先声明，稍后定义
};

// =====
// Box 类
// =====
class Box {
private:
    double length;
    double width;

public:
    Box(double l, double w) : length(l), width(w) {}

    // ✅ 全局函数 做友元
    friend double getArea(const Box& b);

    // ✅ 另一个类的成员函数 做友元
    friend void Printer::print(const Box& b);
};
```

```
};

// ----- 全局友元函数的定义 -----
double getArea(const Box& b) {
    // 直接访问 Box 的私有成员
    return b.length * b.width;
}

// ----- 成员友元函数的定义（必须在 Box 定义之后） -----
void Printer::print(const Box& b) {
    // 直接访问 Box 的私有成员
    cout << "长: " << b.length << ", 宽: " << b.width << endl;
}

// =====
// 主函数
// =====
int main() {
    Box box(5.0, 3.0);

    // 调用全局友元函数
    cout << "面积: " << getArea(box) << endl;

    // 调用成员友元函数
    Printer p;
    p.print(box);

    return 0;
}
```

友元类

若A是B的友元类，那么A的成员函数可以访问B的私有成员

```
#include <iostream>
using namespace std;

// =====
// Box 类
// =====
class Box {
private:
    double length;
```



```

double width;
double height;

public:
    Box(double l, double w, double h)
        : length(l), width(w), height(h) {}

    // ✓ 声明 Inspector 为友元类
    // Inspector 的所有成员函数都能访问 Box 的私有成员
    friend class Inspector;
};

// =====
// Inspector 类 (Box 的友元类)
// =====
class Inspector {
public:
    // 计算体积
    double getVolume(const Box& b) {
        return b.length * b.width * b.height; // ✓ 可访问私有成员
    }

    // 计算表面积
    double getSurfaceArea(const Box& b) {
        return 2 * (b.length * b.width // ✓ 可访问私有成员
                    + b.length * b.height
                    + b.width * b.height);
    }

    // 打印信息
    void printInfo(const Box& b) {
        cout << "==== Box 信息 =====>> endl;
        cout << "长: " << b.length << endl; // ✓ 可访问私有成员
        cout << "宽: " << b.width << endl;
        cout << "高: " << b.height << endl;
        cout << "体积: " << getVolume(b) << endl;
        cout << "表面积: " << getSurfaceArea(b) << endl;
    }
};

// =====
// 主函数
// =====

```

```
int main() {
    Box box(5.0, 3.0, 2.0);

    Inspector inspector;
    inspector.printInfo(box);

    return 0;
}
```

常量成员函数

1. 常量成员函数内部不能改变属性的值，也不能调用非常量成员函数
 - 若变量加上mutable关键字，则此变量在const函数中也能被修改
2. 在**定义和声明**常量成员函数时都应该使用const关键字（写在参数与函数体之间）

```
class Article {
private:
    string text = "这是一篇文章";

    // 加上 mutable，表示这个变量即使在 const 函数中也能被修改
    mutable int readCount = 0;
    int book_number=0;

public:
    // 这是一个常量函数（只读）
    void read() const {
        cout << text << endl; // 正常读取文章

        readCount++; // ✅ 合法！因为 readCount 声明为了 mutable
        //book_number++;不合法，编译会报错
        cout << "文章被阅读了 " << readCount << " 次" << endl;
    }
};
```

常量成员函数的重载:

同名函数，仅通过有无 const 修饰来区分，构成重载。

```
class A {
public:
    void func();           // 非 const 版本
    void func() const;     // const 版本 ← 两者构成重载！
};
```

- 普通对象**优先**调用非const版本
- const 对象**只能**调用const版本

| 与是否重载无关，const对象只能调用const成员函数

对象

定义：类定义出来的变量

内存分配：成员变量大小之和（不包括成员函数）

运算：可以用=赋值，其他需经过运算符重载

使用类的成员变量和成员函数

```
//用法一：对象名.成员名
CRectangle r1,r2;
r1.w=5;
r2.Init(5,4);
//用法二：指针-->成员名
CRectangle r1,r2;
CRectangle * p1= & r1;
p1->w=5;
p1->Init(5,4);
//用法一：引用名.成员名
CRectangle r1;
CRectangle & rr=r1;
rr.w=5;
rr.Init(5,4);
```

const成员和引用成员的初始化

| 在构造函数中必须使用初始化列表，不能在函数体中赋值

```
class Test {
    const int a;
    int& b;
public: // 执行顺序：先执行冒号后的初始化列表，再执行 {} 里的内容
    Test(int val, int& ref) : a(val), b(ref) {
        // 函数体可以为空，或者做其他非初始化操作
    }
};
```